

Rapport technique — traduction-audio

Rapport technique

Traduction-audio : plateforme LLMOps de traduction audio temps réel

Auteur : Tetyana Tarasenko Mentor : Sébastien, DataScientest Date de soutenance : 3 septembre 2026 Repository : github.com/TatianaT13/translate-audio-NLP-Ai
Production : <https://traduction-audio.fr>

Table des matières

1. Résumé
 2. Le besoin auquel je réponds
 3. Ce que j'ai construit
 4. Le paysage technique existant
 5. Ma méthodologie
 6. Architecture d'ensemble
 7. Les choix techniques que j'ai faits
 8. Ingénierie des données et évaluation
 9. Sécurité
 10. Observabilité
 11. Déploiement et intégration continue
 12. Résultats et chiffres
 13. Difficultés rencontrées
 14. Perspectives
 15. Conclusion
 16. Annexes techniques
-

1. Résumé

Traduction-audio est une plateforme de traduction audio temps réel que j'ai conçue, développée et mise en production dans le cadre de mon projet de fin d'études. L'utilisateur envoie un audio en français, la plateforme le transcrit, le traduit dans la langue de son choix (anglais, espagnol, allemand, italien ou ukrainien) et lui renvoie un fichier audio synthétisé dans cette langue.

Le point de départ, ce sont les flash-infos autoroutières de la radio 107.7. Ces alertes trafic sont diffusées uniquement en français, alors que les autoroutes françaises accueillent chaque année plusieurs millions d'utilisateurs étrangers. Un chauffeur ukrainien, un touriste allemand ou un transporteur espagnol ne comprend pas quand la radio annonce un accident, un bouchon ou une déviation.

Techniquement, j'ai choisi une approche LLMOps plutôt que MLOps classique. Je ne réentraîne pas de modèles. Je m'appuie sur des modèles pré-entraînés existants (Whisper pour la reconnaissance vocale, un LLM pour la traduction, un TTS pour la synthèse) que j'orchestre dans une chaîne fiable, observable et sécurisée. L'ensemble tourne dans 14 conteneurs Docker déployés sur un VPS Hetzner, avec du HTTPS, une authentification JWT, du monitoring Prometheus et Grafana, du tracing MLflow et Langfuse, un batch nocturne orchestré par Airflow, et une intégration continue via GitHub Actions.

Trois fonctionnalités principales sont livrées. Le socle est la traduction à la demande depuis un fichier ou le micro. Ce socle est étendu par deux features émergées pendant le développement : un enregistreur de réunion avec compte-rendu automatique, et une traduction simultanée en direct via WebRTC connecté à l'API OpenAI Realtime. Ces deux extensions ouvrent des débouchés produits intéressants au-delà du cas d'usage initial.

2. Le besoin auquel je réponds

Autoroute Info diffuse ses flash-infos sur la fréquence 107.7 en continu, 24 heures sur 24. Les zones couvertes vont de l'Île-de-France à la Bourgogne, en passant par le nord, le sud, l'est et l'ouest. Le contenu est structuré : accidents, bouchons, travaux, animaux sur la chaussée, fermetures de voies, événements météo. Toutes ces informations sont critiques pour la sécurité, mais elles ne sont émises qu'en français.

En pratique, cela crée un déséquilibre. Un usager français peut anticiper une déviation ou ralentir avant un bouchon, tandis qu'un usager étranger conduit sans cette information. Les conséquences sont concrètes : accidents secondaires liés à un ralentissement non anticipé, freinages tardifs sur bouchon, sentiment général d'exclusion des services publics de sécurité routière.

Le défi technique n'est pas anodin. Il ne s'agit pas de traduire des documents statiques mais un flux audio, souvent bruité (moteur, radio, vitres ouvertes), avec des noms propres, des numéros de sorties et un vocabulaire spécifique. Il faut aussi que la chaîne complète soit rapide, autour de deux ou trois secondes de latence, sinon l'information arrive après le bouchon.

Enfin, il faut que cette chaîne soit fiable. Un LLM peut halluciner, un provider cloud peut tomber, un audio peut contenir une tentative d'injection de prompt. Toutes ces préoccupations de production sont au cœur de mon approche LLMOps.

3. Ce que j'ai construit

La plateforme propose trois modes d'utilisation qui répondent à trois cas d'usage distincts.

Le premier mode est la traduction à la demande. C'est le socle du projet, celui qui répond directement au besoin des chauffeurs étrangers. L'utilisateur dépose un fichier MP3 ou WAV sur l'interface, ou bien il enregistre directement au micro depuis son navigateur. Il choisit la langue cible, valide, et reçoit après quelques secondes la transcription en français, la traduction en langue cible, et un fichier audio synthétisé qu'il peut écouter ou télécharger.

Une fois ce socle stabilisé, j'ai démontré sa réutilisabilité avec deux extensions. La première est un enregistreur de réunion qui a émergé pendant le développement. L'utilisateur lance un enregistrement micro long, la plateforme découpe le flux en morceaux de trente secondes, les transcrit à la volée en affichant le texte au fur et à mesure, puis génère à la fin un compte-rendu structuré via un LLM. L'utilisateur choisit entre trois styles de résumé : synthétique pour les décisions, détaillé pour un procès-verbal, ou uniquement les actions à faire. Cette fonctionnalité constitue une piste de monétisation SaaS sur un marché déjà porteur (Otter, Fireflies, Grain).

La seconde extension est la traduction simultanée en direct, via WebRTC. Le navigateur se connecte directement à l'API OpenAI Realtime, en récupérant au préalable un token éphémère auprès de mon serveur (afin de ne jamais exposer ma vraie clé OpenAI côté client). L'utilisateur parle, la traduction sort dans son casque avec environ 500 millisecondes de latence. Cette feature transforme la démonstration : le jury peut littéralement parler en français et entendre l'anglais en temps réel.

En parallèle de ces trois modes, un service backend nommé Watcher tourne en continu. Il interroge périodiquement le flux radio 107.7, transcrit les nouveaux flash-infos avec un Whisper embarqué, extrait les événements structurés (zone, sévérité, type), les traduit dans les cinq langues cibles, et les pousse en Server-Sent Events vers un dashboard administrateur. Cela permet à un opérateur de voir en direct ce qui se passe sur le réseau autoroutier français.

Enfin, le dashboard administrateur est un centre de contrôle LLMOps. On y voit les métriques infrastructure, les expériences MLflow, les DAGs Airflow, la liste des utilisateurs, les traces LLM Langfuse et les coûts cumulés. L'accès est protégé par un rôle `is_admin` sur le compte utilisateur.

4. Le paysage technique existant

Avant de commencer, j'ai regardé ce qui existe. Plusieurs solutions occupent le marché de la traduction et de la reconnaissance vocale, mais aucune ne couvre exactement mon besoin.

Google Translate API est la référence pour la traduction texte, mais c'est une boîte noire : impossible d'ajuster le prompt, de tracer une requête, ou de choisir un modèle plutôt qu'un autre. DeepL propose une bonne qualité de traduction, mais son API n'inclut ni STT ni TTS, ce qui obligerait à composer une chaîne complète manuellement.

AssemblyAI fait du STT en streaming très rapide, mais son tarif à la minute devient prohibitif à volume. L'API Whisper d'OpenAI est simple et précise, mais je n'ai aucun contrôle sur la version du modèle utilisée en coulisses. Azure Speech Translation propose une pipeline complète, mais son SDK est lourd et orienté grande entreprise.

J'ai donc choisi de construire ma propre pipeline en assemblant les briques les plus adaptées à chaque étape. Whisper large-v3 en local pour la transcription, un LLM cloud via LiteLLM pour la traduction (avec fallback multi-provider), Voxtral et MMS-TTS pour la synthèse vocale selon la langue cible. Cette approche me donne la maîtrise complète du coût, de la latence, du prompt, et des versions de modèles.

5. Ma méthodologie

Mon approche s'inscrit dans la démarche LLMOps, qui diffère du MLOps traditionnel sur un point essentiel. En MLOps classique, l'enjeu principal est d'entraîner et de re-entraîner des modèles custom. Ici, je n'entraîne rien. Les modèles sont pré-entraînés et téléchargés. Ce qui me demande de la rigueur, c'est leur orchestration, leur observation et leur mise à jour, dans une chaîne qui doit rester stable en production.

J'ai découpé le projet en quatre phases.

La première phase, de mars à avril 2026, a été consacrée à l'ingénierie des prompts et à la sélection des modèles. J'ai construit un corpus de référence (« golden ») de flash-infos réels transcrits et traduits à la main dans les langues cibles. J'ai ensuite testé douze configurations différentes en croisant deux tailles de modèle Whisper, deux modèles LLM et trois versions de prompt. Chaque configuration a été évaluée sur les sept audios du golden, ce qui donne 84 évaluations individuelles, agrégées en 12 runs MLflow (un par configuration, avec les métriques moyennées sur les sept audios).

La deuxième phase, d'avril à mai, a consisté à découper le système en microservices FastAPI conteneurisés, à mettre en place MLflow comme registre de modèles et Langfuse comme registre de prompts et de traces.

La troisième phase, en mai et juin, a été l'assemblage. J'ai construit le pipeline central en LangChain LCEL, ajouté la gateway avec authentification JWT, et déployé le tout sur mon VPS Hetzner en HTTPS.

La quatrième phase, en juillet et août, a été consacrée au monitoring et à l'évaluation batch. J'ai ajouté Prometheus et Grafana pour les métriques infrastructure, instrumenté le pipeline pour permettre le tracing MLflow et Langfuse, et créé deux DAGs Airflow.

Les trois fonctionnalités bonus (meeting recorder, live WebRTC, watcher radio) ont émergé pendant la phase 4, quand l'infrastructure était suffisamment stable pour permettre d'ajouter des features sans casser l'existant.

6. Architecture d'ensemble

Le système est composé de 14 conteneurs Docker orchestrés par un unique fichier `docker-compose.yml`. Ces 14 conteneurs se décomposent en 6 microservices FastAPI que j'ai développés (gateway, pipeline, stt, llm, tts, watcher), 1 frontend Next.js, et 7 conteneurs d'infrastructure (mlflow, prometheus, grafana, airflow-postgres, airflow-init, airflow-webserver, airflow-scheduler).

À l'entrée, un serveur Nginx écoute sur le port 443. C'est le seul port exposé publiquement. Il termine le TLS avec des certificats Let's Encrypt renouvelés automatiquement par certbot, puis fait un reverse proxy vers le frontend Next.js et l'API gateway. Tous les autres conteneurs sont bindés sur `127.0.0.1` et invisibles depuis l'extérieur.

Le frontend Next.js gère l'interface utilisateur. Il expose huit pages : la page d'accueil avec upload et micro, les pages d'authentification, la page meeting recorder, la page live WebRTC, et le dashboard admin.

La gateway est un service FastAPI qui centralise l'authentification JWT, expose l'API d'administration, et sert de proxy pour la création des tokens éphémères OpenAI Realtime utilisés par le live WebRTC. Dans l'implémentation actuelle, le pipeline reste également accessible depuis le frontend derrière Nginx (les variables `NEXT_PUBLIC_PIPELINE_URL`, `NEXT_PUBLIC_STT_URL`, `NEXT_PUBLIC_LLM_URL` sont bakerisées dans le bundle Next.js). La centralisation complète des appels applicatifs derrière la gateway fait partie du durcissement prévu en phase 2.

Le pipeline est le cœur métier. C'est un service FastAPI qui embarque un orchestrateur LangChain LCEL. À chaque requête `/process`, il enchaîne trois étapes : appel au service STT pour transcrire l'audio, appel au service LLM pour traduire le texte, appel au service TTS pour synthétiser la voix. Chaque étape est un Runnable LangChain composable, ce qui rend la chaîne testable et instrumentable.

Les trois services d'inférence sont indépendants les uns des autres. Le STT utilise Faster-Whisper en version large-v3. Le LLM passe par LiteLLM qui route vers Groq, OpenAI ou Anthropic selon la configuration. Le TTS route vers Voxtral (Mistral) pour les langues majeures ou MMS-TTS (Meta) pour l'ukrainien et l'italien.

En parallèle du pipeline synchrone, le service Watcher tourne en continu et constitue un pipeline distinct. Il a sa propre instance Whisper embarquée, ce qui lui évite un appel HTTP au service STT dédié à chaque cycle. Il appelle directement le service LLM sans passer par le pipeline (il n'a pas besoin de TTS puisqu'il ne produit que du texte structuré).

Côté outillage, MLflow tourne dans son propre conteneur pour le tracking d'expériences et le registre de modèles. Langfuse est utilisé en version cloud pour éviter d'héberger encore un service. Prometheus scrape les métriques toutes les 15 secondes, Grafana affiche les dashboards versionnés en Git. Airflow tourne en trois conteneurs (scheduler, webserver, base Postgres) et exécute les deux DAGs de batch.

7. Les choix techniques que j'ai faits

Chaque outil retenu l'a été après comparaison avec des alternatives. Je détaille ici les choix les plus structurants.

Frontend

J'ai choisi Next.js 15 en mode output: standalone, ce qui produit une image Docker minimale. Le rendu côté serveur est natif, le hot reload en développement est confortable, et l'écosystème React reste le standard de l'industrie. J'ai écarté Streamlit, qui était initialement suggéré : son UX est trop rigide pour un vrai produit, notamment pour gérer un enregistrement micro avec waveform en direct ou une connexion WebRTC. J'ai aussi écarté Vue.js et Angular, moins adaptés à mon besoin.

Backend

FastAPI pour les six microservices. Le support natif d'async/await me permet de gérer plusieurs requêtes simultanément sans bloquer. La validation Pydantic est intégrée, la documentation OpenAPI est générée automatiquement, et l'intégration avec l'écosystème Python IA est naturelle. J'ai écarté Flask (synchrone, sans typage natif), Django (monolithique, trop lourd pour du microservice) et Express.js en Node (obligerait à réécrire toute la logique ML en JavaScript).

Orchestration du pipeline

LangChain LCEL, avec son opérateur | qui compose les étapes comme dans un shell Unix. Chaque étape est un Runnable typé et testable. Le tracing Langfuse est natif à condition d'initialiser un client. J'ai considéré LangGraph, mais mon flow est linéaire (STT puis LLM puis TTS), sans branchement, donc LangGraph serait surdimensionné. J'ai aussi considéré un simple code Python maison, mais je perdrais l'écosystème LangChain (retry, tracing, composition, testing).

Couche LLM

LiteLLM comme couche d'abstraction. C'est un proxy Python qui unifie l'API vers plus de cent providers avec le même format que l'API OpenAI Chat Completions. Il gère les tarifs intégrés et le calcul du coût.

La preuve concrète que ce choix est le bon : en août 2026, Groq a déprécié le modèle que j'utilisais alors en production. La migration vers OpenAI GPT-4o mini a demandé exactement une modification, la valeur de la variable d'environnement LLM_MODEL. Zéro ligne de code touchée dans mes services. C'est exactement ce qu'on attend d'une bonne couche d'abstraction.

Le modèle par défaut en production est actuellement openai/gpt-4o-mini, choisi pour son ratio qualité/coût sur ce cas d'usage. La configuration Docker garde encore le modèle Groq comme valeur par défaut du service LLM, override en production via variable d'environnement. Une harmonisation est prévue.

Reconnaissance vocale (STT)

Faster-Whisper en version large-v3. C'est une implémentation CTranslate2 optimisée, sensiblement plus rapide que le Whisper Python vanille en CPU. Le modèle supporte 99 langues nativement, ce qui me sert aussi pour le mode Live où l'utilisateur peut parler dans une langue autre que le français.

Synthèse vocale (TTS)

Ici j'ai fait un choix hybride selon la langue. Voxtral, le TTS de Mistral, pour le français, l'anglais, l'espagnol et l'allemand. C'est un modèle récent, avec des voix naturelles. Pour l'ukrainien et l'italien, je route vers MMS-TTS de Meta, qui couvre plus de mille langues.

Ce routage par langue m'évite d'avoir à choisir entre qualité (Voxtral) et couverture (MMS). J'ai écarté OpenAI TTS et ElevenLabs pour des raisons de coût et de couverture linguistique.

Conteneurisation

Docker Compose plutôt que Kubernetes. Kubernetes aurait ajouté une complexité opérationnelle injustifiée pour un déploiement mono-VPS. Docker Compose me donne un fichier YAML unique, une commande `up --build`, et des healthchecks natifs.

Reverse proxy

Nginx avec certbot pour Let's Encrypt. Mature, éprouvé, configuration lisible.

Authentification

JWT custom plutôt qu'un service tiers. Les tokens d'accès sont signés en HS256 avec une durée de vie de quinze minutes. Les tokens de refresh sont aléatoires, stockés hashés en SHA-256 en base, et rotatifs à chaque utilisation. Les mots de passe sont hashés avec bcrypt. Ce module est couvert par des tests unitaires qui vérifient le roundtrip, l'expiration, le tampering, les mauvaises signatures et les mauvais algorithmes.

J'ai écarté Auth0, Firebase Auth et Keycloak, chacun apportant une dépendance ou une lourdeur non justifiée pour ce projet.

Orchestration batch

Airflow 2.10, avec deux DAGs. Le premier, `nightly_golden_eval`, tourne tous les jours à 2 heures UTC. Il extrait les audios golden (limité à 7 pour le batch quotidien), appelle le pipeline, agrège les taux de succès, la latence et le coût. Il alerte si trop de runs échouent. L'intégration automatique des métriques de qualité BLEU/METEOR et de l'alerting Slack constitue l'étape suivante prévue.

Le second DAG, `weekly_drift_check`, tourne le dimanche à 3 heures UTC. Il interroge les scores disponibles dans Langfuse pour comparer la semaine N à la semaine N-1 sur la latence, le coût, la probabilité de langue et, lorsqu'il est disponible, le BLEU. Il alerte si la variation dépasse 10%.

MLflow

MLflow est utilisé pour le tracking des expérimentations et pour référencer les modèles externes utilisés par la plateforme. Les 12 configurations sont enregistrées sous forme de runs agrégés (un par configuration unique), chaque run contenant les métriques moyennées sur les 7 audios du golden dataset ; la meilleure configuration est identifiée par le tag `champion=true` accompagné du tag `stage=production`. Trois entrées du Model Registry documentent également les modèles STT, LLM et TTS avec leur version de production. L'évaluation offline est renforcée par `mlflow.evaluate()` qui offre nativement plusieurs métriques dont BLEU, ROUGE et exact match. J'ai enfin instrumenté les steps du pipeline avec des décorateurs `@mlflow.trace` pour permettre le tracing distribué ; son activation en production reste conditionnée à la disponibilité d'un artifact store distant.

En parallèle, 84 évaluations individuelles (12 configurations × 7 audios) sont importées dans Langfuse pour le drill-down par audio.

Prometheus et Grafana

Prometheus scrape le endpoint `/metrics` des six services FastAPI toutes les quinze secondes, via la bibliothèque `prometheus-fastapi-instrumentator`. Grafana affiche les dashboards de latence, throughput, erreurs et coûts. Les dashboards sont versionnés en Git dans `monitoring/grafana/dashboards/`.

Langfuse

Complémentaire à Prometheus, focalisé sur le versant métier LLM. Il capture chaque appel avec l'input, l'output, la latence, le coût, les tokens. La vue waterfall affiche les spans du pipeline dans l'ordre chronologique. Le versioning des prompts est fait de son côté.

Une note importante : Langfuse a publié sa v4 pendant mon développement, avec des breaking changes SDK. J'ai dû refactorer le client complet. Migration transparente pour l'utilisateur final.

8. Ingénierie des données et évaluation

Le corpus golden est composé de flash-infos radio 107.7 réels, chacun accompagné d'une traduction humaine validée. L'ensemble est stocké dans `data/golden/` et versionné en Git. Le benchmark de la phase 1 utilise 7 audios de ce corpus.

Pour la phase de sélection, j'ai construit un plan d'expérience à 12 configurations : deux modèles Whisper (small pour la rapidité, large-v3 pour la qualité), deux modèles LLM (Llama 3.1 8B et Llama 3.3 70B via Groq à l'époque), et trois versions de prompt (v1.0 basique, v1.1 pro traffic, v1.2 broadcast quality).

Chaque configuration a été évaluée sur les 7 audios du golden. Cela donne 84 évaluations individuelles importées dans Langfuse pour le drill-down audio par audio, et 12 runs agrégés dans MLflow (un par configuration, avec les métriques moyennées sur les 7 audios). Cette organisation me permet de comparer les configurations en un coup d'œil dans MLflow tout en pouvant revenir sur une évaluation individuelle dans Langfuse.

Les métriques utilisées sont classiques en évaluation de traduction. BLEU (sacrebleu) mesure la similarité en n-grams avec la traduction humaine de référence. METEOR (nltk) est une version pondérée qui prend en compte les synonymes. WER (jiwer) mesure la qualité STT.

Le champion expérimental identifié par cette campagne est la configuration `large-v3 + Llama 3.3 70B + prompt v1.1`, avec un BLEU moyen de **49.64** et un METEOR moyen de **0.713** sur les 7 audios du golden.

Historiquement, j'ai retenu en production le modèle Llama 8B pour son compromis coût/performance, la qualité du 70B ne justifiant pas le surcoût sur ce cas d'usage précis (des flash-infos courts et structurés).

Suite à la dépréciation de `llama-3.1-8b-instant` par Groq en août 2026, la production a été migrée vers OpenAI GPT-4o mini via LiteLLM (variable d'environnement, aucune ligne de code touchée). Cette configuration n'a pas encore été rebenchmarkée sur le même protocole que la campagne initiale : une nouvelle campagne comparative sur les 7 audios du golden est prévue pour valider scientifiquement le maintien de la qualité.

L'évaluation continue est automatisée par le DAG Airflow `nightly_golden_eval`. Il tourne toutes les nuits, ping le pipeline pour vérifier sa disponibilité, appelle le pipeline sur les 7 audios du golden, agrège les taux de succès, la latence et le coût. Il alerte si au moins la moitié des runs échouent. L'intégration automatique du calcul BLEU/METEOR dans le DAG et l'alerting Slack constituent l'étape suivante prévue.

Le DAG `weekly_drift_check` complète ce dispositif en comparant la semaine courante à la semaine précédente sur la latence, le coût, le BLEU et la probabilité de langue, avec un seuil de variation de 10%.

9. Sécurité

La sécurité est traitée en trois couches, selon le principe de défense en profondeur.

La couche réseau isole tout ce qui peut l'être. Les quatorze conteneurs Docker sont bindés sur 127.0.0.1 et invisibles depuis Internet. Seul Nginx expose le port 443 en HTTPS. Les certificats Let's Encrypt sont renouvelés automatiquement par certbot. Cette isolation réduit fortement leur surface d'exposition en empêchant leur accès direct depuis Internet.

La couche authentification applicative repose sur du JWT custom. Les tokens d'accès sont signés en HS256 avec une durée de vie courte de quinze minutes. Les tokens de refresh sont plus longs (sept jours) mais hashés en SHA-256 en base, et surtout rotatifs (l'ancien est révoqué à chaque utilisation). Les mots de passe sont hashés avec bcrypt en salt automatique. Ce module est couvert par 25 tests unitaires qui vérifient le roundtrip, l'expiration, le tampering, les mauvaises signatures et les mauvais algorithmes.

La couche LLM est celle qui m'a demandé le plus d'attention, parce qu'elle est la plus spécifique. La menace principale est l'injection de prompt, cataloguée OWASP LLM01. Un audio malicieux pourrait contenir des instructions cachées comme « ignore previous instructions » ou « reveal your system prompt ». J'ai mis en place trois mesures de mitigation inspirées d'OWASP LLM01. D'abord un pre-check regex bilingue (français et anglais) qui détecte les patterns d'injection connus et bloque la requête avant même qu'elle n'atteigne le LLM. Ensuite une séparation structurée des données utilisateur dans le prompt, qui échappe les balises XML pour éviter qu'un contenu utilisateur soit interprété comme instruction système. Enfin un post-check qui vérifie le ratio de longueur input/output (une hallucination produit souvent un output démesuré à partir d'un input court) et détecte les prompt leaks classiques. Ce module est couvert par 24 tests unitaires.

10. Observabilité

J'ai adopté une approche à trois niveaux, complémentaires les uns des autres.

Le niveau infrastructure est couvert par Prometheus et Grafana. Prometheus scrape les métriques HTTP standards (requêtes par seconde, latence, erreurs 4xx et 5xx) sur les six services FastAPI, plus les métriques système. Les dashboards Grafana sont organisés par service et par percentile de latence. Cette couche répond aux questions ops : est-ce que le service est up, combien de requêtes traite-t-il, quelle est la latence p95.

Le niveau applicatif est couvert par une instrumentation MLflow prête à l'emploi. J'ai décoré les steps du pipeline avec `@mlflow.trace` : lorsque le tracing est activé, chaque appel produit un span parent avec des sous-spans hiérarchisés, visibles dans l'UI MLflow avec le temps consommé par chaque step. L'activation en production reste conditionnée à la disponibilité d'un artifact store distant.

Le niveau métier LLM est couvert par Langfuse. Cet outil est spécialisé dans l'observabilité des applications LLM. Il capture chaque appel avec l'input, l'output, la latence, le coût, les tokens consommés. La vue waterfall affiche les spans dans l'ordre chronologique. Les prompts sont versionnés (v1.0, v1.1, v1.2).

Les trois couches se recoupent partiellement mais couvrent des besoins différents. Prometheus me dit qu'un service est lent, MLflow me dit à quelle étape le ralentissement se produit, Langfuse me dit combien coûte cette lenteur.

11. Déploiement et intégration continue

La production tourne sur un VPS Hetzner sous Ubuntu Server. Le domaine `traduction-audio.fr` est géré par OVHcloud, avec le HTTPS assuré par Let's Encrypt et le renouvellement automatique via certbot.

L'intégration continue tourne sur GitHub Actions. À chaque push ou pull request sur la branche `main`, le CI exécute automatiquement les tests unitaires (`pytest tests/unit/`) et publie le rapport JUnit en artifact. Les tests d'intégration sont exécutés séparément car ils nécessitent les services Docker en local. Un échec bloque le merge de la branche.

Le déploiement continu est en place via un second workflow qui se déclenche automatiquement après un CI réussi sur `main`. Il utilise SSH pour se connecter au serveur, synchronise `origin/main`, appelle le script `scripts/deploy.sh` qui rebuild les images sans cache et force-recreate les containers, puis affiche l'état des conteneurs et de leurs healthchecks via `docker compose ps`. Un déclenchement manuel est aussi possible via l'interface GitHub Actions.

Les tests sont organisés en trois catégories. La suite unit couvre les modules critiques : authentification JWT, chunking de long audios, routing TTS par langue, alias de modèles, prompt-guard, résistance aux injections. La suite intégration couvre les flows bout-en-bout : login, pipeline complet, appels HTTP entre services. Une suite e2e est prévue en phase 2, avec Playwright pour tester l'interface Next.js.

12. Résultats et chiffres

Sur la qualité de traduction, le champion expérimental est la configuration `large-v3 + Llama 3.3 70B + v1.1` avec un score BLEU moyen de **49.64** et un score METEOR moyen de **0.713** sur le corpus golden. Ce champion représente le meilleur compromis qualité observé lors de la campagne de sélection.

La configuration Llama 8B avait historiquement été retenue en production pour son compromis coût/performance, malgré une qualité inférieure au champion 70B. Suite à la dépréciation Groq d'août 2026, la production actuelle utilise OpenAI GPT-4o mini via LiteLLM. Cette configuration n'a pas encore été rebenchmarkée sur le même protocole, une nouvelle campagne est prévue.

Sur la performance, la latence end-to-end oscille entre deux et trois secondes selon la longueur de l'audio. Le détail est le suivant : environ 800 millisecondes pour le STT (Whisper large-v3 en CPU), 1,2 seconde pour le LLM, 600 millisecondes pour le TTS. Le mode Live WebRTC est nettement plus rapide avec 500 millisecondes de latence first-byte, puisque OpenAI Realtime fait tout en interne.

Sur le coût, une traduction upload revient à moins de 0,001 dollar en production. Une session live coûte environ 0,30 dollar par minute (OpenAI Realtime est plus cher). Le coût mensuel d'infrastructure du VPS est d'environ 30 euros par mois.

Sur les volumes, le projet compte 12 runs MLflow (agrégés sur les 7 audios), 84 évaluations individuelles dans Langfuse, 14 conteneurs Docker en production (6 microservices + 1 frontend + 7 conteneurs d'infrastructure), 24 routes API sur la gateway, 8 pages frontend, 6 langues supportées (français source, plus anglais, espagnol, allemand, italien, ukrainien en cible).

13. Difficultés rencontrées

Je détaille ici les problèmes les plus significatifs que j'ai dû résoudre, parce qu'ils illustrent bien les défis d'un projet LLMOps réel.

La migration de Langfuse v2 vers v4 a été mon plus gros refactor. La v4 a introduit un nouveau modèle avec context managers, et certains arguments ont été déplacés du kwargs vers le dictionnaire metadata. Toute mon instrumentation du pipeline a dû être réécrite. Une bonne demi-journée de travail, mais la migration est transparente pour l'utilisateur.

La dépréciation successive par Groq du modèle llama-3.1-8b-instant puis de gpt-oss-20b en été 2026 a provoqué des erreurs 500 en cascade dans le pipeline. J'ai résolu ça avec un pattern de model aliases côté serveur : un dictionnaire qui redirige transparentement les anciens noms de modèles vers OpenAI GPT-4o mini. Aucun changement côté client. Ce cas concret a validé rétroactivement mon choix architectural de LiteLLM comme couche d'abstraction.

L'enregistreur de meeting a eu un bug subtil : les chunks 2 et suivants revenaient transcrits comme vides. La cause était dans la logique du MediaRecorder du navigateur : appeler `start(30_000)` avec un timeslice génère des fragments WebM sans header, et Whisper decode correctement le premier fragment mais échoue silencieusement sur les suivants. La solution a été de passer sur un pattern stop+restart du MediaRecorder : chaque cycle stop produit un fichier WebM complet avec header, et je redémarre immédiatement un nouveau cycle.

Le mode Live WebRTC a eu un problème physique de feedback loop. Sans casque, le speaker rejoue la traduction dans le micro, le modèle OpenAI se re-traduit lui-même, et on se retrouve dans une boucle infinie de délires. La solution logicielle a été de mute automatiquement le track micro dès que le modèle commence à parler (`event response.audio.delta`), puis de le unmute à la fin (`response.done`). Le feedback loop devient physiquement impossible même sans casque.

Le cookie de session Next.js avait un TTL de quinze minutes alors que le refresh JWT dure sept jours. Résultat : au bout de quinze minutes d'inactivité, l'utilisateur était redirigé vers `/login` alors qu'il pouvait encore rafraîchir son token

silencieusement. J'ai aligné la durée du cookie sur celle du refresh (sept jours). Le cookie ne sert que d'indicateur de session pour le middleware Next.js ; la vraie validation JWT reste faite par la gateway sur chaque appel API.

Whisper avait tendance à halluciner sur les accents. Sans hint de langue explicite, un français avec un accent était parfois transcrit dans la mauvaise langue. La solution a été de passer explicitement `language: "fr"` dans la config transcription, et d'ajouter dans le mode Live un sélecteur « Je parle en » qui permet à l'utilisateur de forcer une autre langue source.

14. Perspectives

Le projet est livré en phase 1. La phase 2 comporte plusieurs chantiers.

Côté qualité et évaluation, la priorité est d'intégrer le calcul BLEU et METEOR dans le DAG nightly, qui contrôle actuellement la disponibilité du pipeline et exécute le golden dataset afin de suivre les taux de succès, la latence et le coût, et de brancher un alerting Slack automatique. Une nouvelle campagne comparative doit aussi être menée sur OpenAI GPT-4o mini pour valider scientifiquement la qualité de la configuration de production actuelle sur le même protocole que la campagne initiale.

Côté architecture, la centralisation complète des appels applicatifs derrière la gateway est prévue, en remplacement des URLs de services directs actuellement bakerisées dans le bundle Next.js. Cela renforce l'isolation réseau et facilite le scaling futur.

Côté outillage, j'aimerais ajouter MinIO comme stockage S3-like pour préparer un éventuel scaling multi-instance. Ragas apporterait des métriques d'évaluation LLM plus avancées, même si le use case traduction le rend moins critique que dans une pipeline RAG. Evidently permettrait une détection de drift au niveau des features. Une pyannote.audio pourrait ajouter de la diarization (identification du locuteur) au meeting recorder, pour un compte-rendu nominatif du type « Alice a dit » et « Bob a proposé ». Enfin, l'harmonisation des valeurs par défaut Docker entre le frontend et le service LLM est prévue.

Côté produit, deux débouchés se dessinent. Un débouché B2B via un partenariat avec un opérateur autoroutier (Vinci, APRR, ATMB) pour intégrer traduction-audio dans leurs applications mobiles usagers. Un débouché B2C via une offre freemium sur le meeting recorder, qui a un vrai potentiel de monétisation SaaS. Il y a aussi la question de l'extension linguistique : passer de six à quinze langues européennes est faisable puisque Whisper les gère nativement.

Sur le plan des compétences, ce projet me valide un ensemble transférable : Python asynchrone avec FastAPI, TypeScript et React avec Next.js 15, Docker Compose multi-service, l'écosystème LLMops complet (LangChain, LiteLLM, MLflow, Langfuse), les fondamentaux DevOps (Nginx, Let's Encrypt, GitHub Actions, VPS deployment) et la sécurité applicative (JWT, bcrypt, prompt-guard).

15. Conclusion

Ce projet démontre qu'il est possible aujourd'hui de construire une plateforme LLMOps complète, sécurisée et déployée en production réelle, en s'appuyant sur des modèles pré-entraînés et un écosystème d'outils matures. L'enjeu n'est pas la modélisation, mais l'orchestration.

Le projet couvre trois niveaux d'orchestration articulés autour d'un socle unique. LangChain LCEL orchestre le temps réel, chaînant STT, LLM et TTS à chaque requête utilisateur. Airflow orchestre le batch, avec l'évaluation nocturne et la surveillance hebdomadaire des dérives. Docker Compose orchestre le cycle de vie de l'ensemble des 14 conteneurs. Cette articulation à trois niveaux est ce qui distingue une vraie plateforme d'un simple script.

L'observabilité est également à trois niveaux complémentaires. Prometheus et Grafana pour l'infrastructure, MLflow pour l'expérimentation et le tracing distribué, Langfuse pour l'observabilité métier LLM (prompts, coûts, tokens, feedback).

Le projet livre plus que ce qui était prévu au départ. Une fois le socle LLMOps stabilisé, j'ai démontré sa réutilisabilité avec deux extensions : un meeting recorder avec résumé LLM automatique et une traduction speech-to-speech temps réel via WebRTC. Ces extensions ouvrent des débouchés produits concrets au-delà du cas d'usage initial.

Il reste bien sûr des choses à améliorer, dont plusieurs sont listées dans les perspectives : intégration réelle des métriques BLEU/METEOR dans le DAG nightly, alerting Slack automatique, centralisation complète des appels applicatifs derrière la gateway, nouvelle campagne comparative sur GPT-4o mini. Mais la fondation est solide et le socle technique est prêt à accueillir une phase 2 ambitieuse.

16. Annexes techniques

Structure du repository

```
translate-audio-NLP-Ai/
├── backend/services/      # 6 microservices FastAPI
│   ├── gateway/          # Auth JWT + Admin API
│   ├── pipeline/         # Orchestrateur LangChain LCEL
│   ├── stt/              # Faster-Whisper large-v3
│   ├── llm/              # LiteLLM multi-provider
│   ├── tts/              # Voxtral + MMS-TTS
│   └── watcher/          # Poll radio 107.7
├── frontend/             # Next.js 15 standalone
├── airflow/dags/          # 2 DAGs Python
├── monitoring/           # Prometheus + Grafana configs
├── scripts/              # Scripts métier
├── tests/                # Suite unit + integration
├── docs/                 # ARCHITECTURE + RUNBOOK + rapport
├── data/golden/          # Audios de référence
└── docker-compose.yml    # 14 services orchestrés
```

Documentation associée

Le repository contient plusieurs documents complémentaires. Le README.md donne un quickstart et un overview technique. Le docs/ARCHITECTURE.md détaille la vue technique. Le docs/RUNBOOK.md décrit les procédures opérationnelles. Le docs/architecture-schema.txt fournit un schéma ASCII complet en fichier texte. Le docs/soutenance.html est le support de soutenance interactif.

Commandes utiles

Pour un setup local complet :

```
git clone git@github.com:TatianaT13/translate-audio-NLP-Ai.git
cd translate-audio-NLP-Ai
cp .env.example .env
# renseigner les clés API dans .env
docker compose up -d --build
```

Pour lancer les tests :

```
pytest tests/unit/           # tests unitaires
pytest tests/integration/    # tests d'intégration (nécessite
                             services Docker)
```

Pour déployer en production :

```
./scripts/deploy.sh          # rebuild tous les services
./scripts/deploy.sh frontend # rebuild uniquement le frontend
```

Pour lancer une évaluation batch manuelle :

```
python scripts/eval_golden.py
python scripts/import_metrics_to_langfuse.py
```

Contacts

Auteur : Tetyana Tarasenko Mentor : Sébastien (DataScientest) Providers LLM :
Groq, OpenAI, Anthropic Hébergement : Hetzner (VPS) et OVHcloud (DNS)

Rapport rédigé pour la soutenance du 3 septembre 2026.